

MODEL REGRESSION DETECTION SYSTEM

A CI/CD Pipeline That Catches LLM Quality Regressions Before They Ship

Project 1 of the AI Engineering Portfolio | Build Guide & Interview Companion
Author: Anas Amiar

"Treat prompt changes like code changes — run them against a known-good answer key before they ever reach production."

How to Use This Guide

This guide documents the Model Regression Detection System exactly as it was built: a short pitch, the tech stack, a phase-by-phase build order with reasoning for why each phase comes before the next, and the talking points worth memorizing before an interview.

Read top to bottom to learn the project from scratch, or jump to the "Interview Talking Point" box at the end for a quick pre-call refresher.

What You're Building

A CI pipeline for LLM-powered features. Whenever a prompt or model change is proposed in a pull request, the system automatically runs the feature against a hand-verified golden dataset, scores it, compares the result to a committed baseline, and blocks the merge if accuracy drops by 8% or more — exactly the way a broken unit test blocks a bad code change.

The feature under test is a customer-support email classifier (billing / technical / account / general). The eval pipeline is generic: any LLM feature with deterministic expected outputs can be plugged in.

WHY THIS PROJECT LANDS INTERVIEWS

Most teams change LLM prompts the way they'd fix a typo — push it, see if anyone complains. There is no equivalent of a unit test suite for "does this prompt still work correctly." This project is that test suite. It demonstrates a production mindset that most candidates don't show: treating AI behavior as something that needs to be tested, versioned, and gated — not just deployed and hoped for the best.

Tech Stack

Layer	Tool / Library	Purpose
Language	Python 3.11+	Core implementation
Data validation	Pydantic v2	Typed contracts for test cases, run results, diffs
Classifier (mock)	Keyword-based rules	Deterministic, no API key needed for full demo
Classifier (real)	OpenAI API (GPT-4o)	Pluggable real-model mode
Prompt management	YAML file	Versioned system prompt + few-shot examples
Reporting	Static HTML	Self-contained scorecard per run, zero setup to open
Alerting	Slack webhook	Warning/critical alerts; dry-run prints locally if no webhook
CI/CD	GitHub Actions	Runs automatically on every relevant PR
Storage	JSON (baseline, history)	Flat-file, no database required

The Core Flow

You change **classifier.py** or a file in **prompts/**. You open a pull request. GitHub Actions runs ``python3 run_eval.py`` automatically. The eval engine scores all 10 golden test cases, compares the pass rate to ``reports/baseline.json``, and exits with a failing status on critical regressions — blocking the merge. If the change is intentional and correct, run ``python3 update_baseline.py`` to promote it.

Step-by-Step Build Guide

Seven phases, in the order they were actually built. Each phase exists because the next one needs its output to work — there's no point building a diff engine before you have a baseline to compare against.

PHASE 1: The Feature Under Test

Day 1

Goal: Have a real AI feature to evaluate — mock-first so everything else can be built without API keys.

- Build ``classifier.py`` with two modes: mock (keyword rules) and real (OpenAI API call).
- Mock mode is intentionally simple so it has real, visible blind spots — non-English input, sarcasm — which gives the eval system genuine regressions to catch.
- Versioned prompt file (``prompts/v1.yaml``) with system prompt + few-shot examples, so prompt changes appear as a clean git diff.

Deliverable: ``classify(email_text, use_mock=True)`` returning category + summary — a real feature that can fail in real ways.

PHASE 2: The Golden Dataset

Day 1

Goal: Build the answer key the entire eval system will measure against.

- Write 10 test emails by hand — never LLM-generated. The whole point is verified ground truth.
- Span three difficulty levels: easy (clear, unambiguous), medium (tricky wording), hard (one-word email, non-English, sarcasm, multi-category).
- Each case has ``expected_category`` and ``expected_summary`` as typed, committed fields.

Deliverable: ``golden_dataset/v1.json`` — a fixed, human-verified answer key the system can always be scored against.

PHASE 3: The Eval Engine

Day 2

Goal: Run the feature against every test case and produce a scored result.

- Runner (``eval_engine/runner.py``): passes each test case through the classifier, records the actual output, scores category (exact match) and summary (word-overlap heuristic).
- Only category affects pass/fail for now — summary quality is visible but advisory.
- Output is a typed ``RunResult``: per-case outcomes + overall pass rate.

Deliverable: ``run_eval(use_mock)`` returning a ``RunResult`` with overall pass rate and per-case detail.

Goal: Make comparisons meaningful by anchoring them to a committed fixed point, not local run history.

- Baseline is a committed `reports/baseline.json` — not a local timestamped file. This is what makes CI work: a fresh GitHub Actions machine has no memory of past runs, but git does.
- Diff logic classifies delta as: ok (less than 3% drop), warning (3-8% drop), critical (8%+ drop, exits non-zero, blocks merge).
- `update_baseline.py` is a separate, deliberate command — baseline never updates automatically.

Deliverable: `compare(current_run, baseline)` returning severity (ok/warning/critical) and the delta.

PHASE 5: Reporting + Alerting

Day 3

Goal: Make failures visible — to a developer reading locally and to a team reading Slack.

- HTML scorecard (``eval_engine/report.py``): one self-contained file per run, green/red per test case, no server needed to open.
- Slack alert (``eval_engine/alerts.py``): sends a warning/critical message with the delta and per-case breakdown. Dry-runs locally if no webhook is configured.

Deliverable: A ``reports/`` folder with an HTML report after every run, and a Slack alert on regressions.

PHASE 6: GitHub Actions CI

Day 3

Goal: Make the gate automatic — no developer needs to remember to run the eval.

- Workflow triggers on PRs that touch ``classifier.py``, ``prompts/``, or ``golden_dataset/``.
- Runs ``python3 run_eval.py``, which exits non-zero on critical regressions.
- Non-zero exit = CI check fails = merge is blocked.
- Slack webhook is passed in as a GitHub Actions secret, so it fires on CI without committing credentials.

Deliverable: ``.github/workflows/eval.yml`` — a fully wired CI gate, verified by deliberately breaking the classifier and watching the PR check fail.

PHASE 7: End-to-End Demo

Day 3-4

Goal: Prove the gate works by running the full lifecycle on a real PR.

- Break the classifier on purpose (degrade the prompt or mock logic).
- Open a PR, watch CI flag the regression, see the HTML report and Slack alert fire.
- Fix the classifier, push to the same PR, watch CI go green.
- Merge — demonstrating a gate that catches regressions AND unblocks once they're fixed.

Deliverable: A live PR showing the full regression-detect-fix-merge cycle.

The Hardest Bug

The first version kept a folder of timestamped run results and compared against "whatever ran last." That worked locally, but in GitHub Actions every run starts on a brand-new machine with no memory of past runs — CI kept reporting "first run, nothing to compare" even when a real regression was sitting right there, completely silent. The fix: commit one ``baseline.json`` to git and make updating it a deliberate, separate step. This is the kind of bug that only appears when you actually wire up CI instead of just trusting the logic "should" work.

INTERVIEW TALKING POINT

"What was the hardest design decision?"

Deciding what counts as ground truth. It would have been easy to use an LLM to generate the golden dataset's expected answers — but then the eval system would just be checking whether one model agrees with another, not whether the feature is actually correct. Writing all 10 test cases by hand, including intentionally hard edge cases like sarcasm and non-English input, was slower but is what makes the pass/fail signal trustworthy. An eval system is only as good as the answer key it measures against.

What I'd Build Next

- LLM-as-judge scoring for summary quality, replacing the current word-overlap heuristic with something more semantically aware.
- Drift detection across a rolling window of runs, not just single-run vs. baseline.
- Statistical significance testing so small, noisy deltas don't trigger false alarms.

Companion Projects in This Portfolio

This project pairs naturally with the **LLM Cost Autopilot** (Project 2), which optimizes cost on every request. Together they cover the two main levers an AI engineering team tunes in production: correctness and cost. The **Failure Forensics Tool** (Project 3) extends the observability angle — diagnosing why a pipeline failed, not just whether it did.

Repository: github.com/Anas-Amiar/Project-1-model-regression-detector